

ASE WEEK 02 — UNIT 09

Choosing a Model

No model wins outright — the three factors that drive selection, the two drivers that dominate, and why the final documentation always reads like waterfall

Sources. Mall, *Fundamentals of Software Engineering* 4th ed., §2.6 p.118–119 (comparison of all models and the customer viewpoint), §2.6.1 p.120 (the three selection factors), Summary p.121 (why final documentation reads as waterfall). Sommerville, *Software Engineering* 9th ed., p.46 (plan-driven versus agile; critical versus business systems). **Note on method.** This is a compendium, not a summary. Every definition is quoted verbatim from the source before it is explained, and every claim carries a page anchor. The selection decision is covered entirely by Mall plus one Sommerville page — Pressman and Agarwal add nothing here and are not cited. **Page convention.** Page numbers are PDF page numbers (PDF index + 1), not printed page numbers — Mall carries no printed page labels, and the front-matter offset differs between the two books.

Where this sits

Unit Context: Unit 08 (Agile, XP, Scrum) ended on its own admission: Agile is **not** the final answer either — it breaks on **critical / safety systems** where a complete upfront analysis is mandatory (Sommerville p.74). The narrative spine therefore turns the question from "*which model is best?*" into "*which model fits?*" (Unit 08 hand-off: "the last question is not 'which is best' but 'which fits'.")

The question this unit answers: *ten models, no winner — so how does a real project lead actually decide?*

The question it hands to the next unit: *now that we can choose, put every model side by side and check what was actually retained.*

The connective tissue: Units 02–08 each answered *one* response to the problem of changing requirements (freeze them → waterfall; learn them → prototype; slice them → incremental; evolve them → evolutionary; compress them → RAD; risk-drive them → spiral; abandon the plan → agile). Unit 09 is the **meta-step**: it gives the *decision rule* that sits above all of them. The thread from Sommerville's own chapter objective — "*why processes should be organized to cope with changes in the software requirements and design*" (p.44) — reaches its conclusion here: you cope by **picking the model whose shape matches your requirements and your risk**.

1. The Central Claim — No Model Wins, Only Fits

Mall opens the comparison chapter by making the hierarchy explicit: the classical waterfall model is the **root**, and everything else is a variation on it.

Verbatim (Mall p.118):

The classical waterfall model can be considered as the basic model and all other life cycle models as embellishments of this model. However, the classical waterfall model cannot be used in practical development projects, since this model supports no mechanism to correct the errors that are committed during any of the phases but detected at a later phase. This problem is overcome by the iterative waterfall model through the provision of feedback paths.

الفكرة الجوهرية: Waterfall الكلاسيكي هو الأصل، وكل النماذج الثانية "تحسينات" (embellishments) عليه. بس هو مو عملي لأنه ما عنده آلية لإصلاح الغلط لو انكشف بمرحلة لاحقة — وهذا اللي يحله Waterfall Iterative بالfeedback paths. يعني حتى "الأصل" نفسه ينتقد ويتطور.

Then the key "no winner" statement — the most-used model is also the most limited:

Verbatim (Mall p.118):

The iterative waterfall model is probably the most widely used software development model so far. This model is simple to understand and use. However, this model is suitable only for well-understood problems, and is not suitable for development of very large projects and projects that suffer from large number of risks.

Waterfall Iterative الأكثر استخداماً والأسهل، بس قيده واضح: ينفع بس للمشاكل المفهومة جيداً، ما ينفع للمشاريع الضخمة أو اللي تعاني من مخاطر كثيرة. أي يعني: ما أكو نموذج "منتصر" — حتى الأكثر شيوعاً له حدود.

This is why Unit 09's title is "No winner, only fit": **being widely used ≠ being universally right.**

2. Mall's Side-by-Side Comparison of All Models (§2.6)

Mall then gives, per model, the **exact situation each one is built for**. The table below is a **paraphrase/derivation from §2.6 (p.118–119)** — the verbatim anchors follow it so every cell is traceable.

Model	Mall's stated suitability (§2.6, p.118–119)
Classical waterfall	The "basic model"; all others are embellishments. Cannot be used in practice (no cross-phase error correction).
Iterative waterfall	Most widely used; simple. Suitable only for well-understood problems ; not for very large or high-risk projects.
Prototyping	Requirements or technical aspects not well understood, but all risks identifiable upfront . Especially popular for the UI part.
Evolutionary	Large problems decomposable into modules for incremental dev/delivery; widely used for object-oriented projects; only usable if the customer accepts incremental delivery.
Spiral	A meta-model that <i>encompasses all other</i> models; flexibility + risk handling built in; for technically challenging, large, risk-prone software; much more complex than the others (a deterrent for ordinary projects).

Verbatim (Mall p.119) — prototyping:

The prototyping model is suitable for projects for which either the user requirements or the underlying technical aspects are not well understood, however all the risks can be identified before the project starts. This model is especially popular for development of the user interface part of projects.

Verbatim (Mall p.119) — evolutionary:

The evolutionary approach is suitable for large problems which can be decomposed into a set of modules for incremental development and delivery. This model is also used widely for object-oriented development projects. Of course, this model can only be used if incremental delivery of the system is acceptable to the customer.

Verbatim (Mall p.119) — spiral as meta-model:

The spiral model is considered a meta model and encompasses all other life cycle models. Flexibility and risk handling are inherently built into this model. The spiral model is suitable for development of technically challenging and large software that are prone to several kinds of risks that are difficult to anticipate at the start of the project. However, this model is much more complex than the other models—this is probably a factor deterring its use in ordinary projects.

خلاصة المقارنة: كل نموذج مصمم لموقف معين. الـ Prototyping للتي مطلوباته/تقنيته مو واضحة بس المخاطر معروفة مقدماً (وينفع للـ UI). الـ Evolutionary للمشاريع الكبيرة القابلة للتقسيم (وخصوصاً OO) وشرطه العميل يوافق على التسليم التدريجي. والـ Spiral ميتا-نموذج يغطي الكل لكنه الأعقد — وهذا يخليه ثقیل على المشاريع العادية.

2.1 The Prototyping-vs-Spiral Distinction (the risk test)

This is the single most exam-relevant comparison in §2.6. Both handle risk; the difference is **when you can see the risk**.

Verbatim (Mall p.119):

Let us now compare the prototyping model with the spiral model. The prototyping model can be used if the risks are few and can be determined at the start of the project. The spiral model, on the other hand, is useful when the risks are difficult to anticipate at the beginning of the project, but are likely to crop up as the development proceeds.

القاعدة: المخاطر معروفة من البداية؟ → Prototyping يكفي. المخاطر تطلع أثناء التطوير؟ → Spiral لازم (لأنه مصمم يكون مرن مع المخاطر اللي تظهر لاحقاً). هذا يربط مباشرة بملف 06 (risk-driven = Spiral).

3. From the Customer's Viewpoint — Confidence, Trauma, Capital (§2.6)

Mall adds a dimension the technical comparison misses: **how the model feels to the customer**. This is why "fit" is not purely technical.

Verbatim (Mall p.119):

Initially, customer confidence is usually high on the development team irrespective of the development model followed. During the lengthy development process, customer confidence normally drops off, as no working software is yet visible. Developers answer customer queries using technical slang, and delays are announced. This gives rise to customer resentment. On the other hand, an evolutionary approach lets the customer experiment with a working software much earlier than the monolithic approaches. Another important advantage of the incremental model is that it reduces the customer's trauma of getting used to an entirely new system. The gradual introduction of the software via incremental phases provides time to the customer to adjust to the new software. Also, from the customer's financial view point, incremental development does not require a large upfront capital outlay. The customer can order the incremental versions as and when he can afford them.

من وجهة نظر العميل: Waterfall (الـ monolithic) **الثقة تهبط** لأن ما أكو برنامج شغال يُرى، والمطورين يردون بلغة تقنية والعميل ينزع. Incremental/Evolutionary: العميل **يتجرب البرنامج باكراً، ويتعود تدريجياً** (أقل trauma)، وما يحتاج رأس مال **ضخم مقدماً** — يطلب الزيادات لما يقدر يدفع. هذا سبب "تجاري" قوي لاختيار النماذج التزايدية.

Synthesis [Foundational Knowledge / Standard Concept]: "Fit" has three faces — technical (does the model match the problem?), human (does it keep the customer confident?), and financial (can the customer afford the cash-flow shape?). A model can be technically perfect and still be the wrong fit if it breaks customer confidence or cash flow.

4. The Three Selection Factors — Product, Team, Customer (§2.6.1)

Mall now answers *how to choose* directly. The answer is **three factors**.

Verbatim (Mall p.120) — the framing:

We have discussed the advantages and disadvantages of the various life cycle models. However, how to select a suitable life cycle model for a specific project? The answer to this question would depend on several factors. A suitable life cycle model can possibly be selected based on an analysis of issues such as the following:

السؤال المباشر: كيف تختار؟ الجواب عند Mall يعتمد على **ثلاثة عوامل**: طبيعة المنتج، طبيعة فريق التطوير، وطبيعة العميل.

Factor 1 — Characteristics of the software (the product)

Verbatim (Mall p.120):

Characteristics of the software to be developed: The choice of the life cycle model to a large extent depends on the nature of the software that is being developed. For small services projects, the agile model is favoured. On the other hand, for product and embedded software development, the iterative waterfall model can be preferred. An evolutionary model is a suitable model for object-oriented development projects.

طبيعة المنتج: خدمات صغيرة → Agile → منتج/embedded → Iterative Waterfall → OO → Evolutionary.

Factor 2 — Characteristics of the development team

Verbatim (Mall p.120):

Characteristics of the development team: The skill-level of the team members is a significant factor in deciding about the life cycle model to use. If the development team is experienced in developing similar software, then even an embedded software can be developed using an iterative waterfall model. If the development team is entirely novice, then even a simple data processing application may require a prototyping model to be adopted.

مهارة الفريق تقلب القرار: فريق خبير حتى الـ embedded يكدر يسويه Waterfall Iterative. فريق مبتدئ تماماً حتى أبسط تطبيق معالجة بيانات قد يحتاج Prototyping. يعني نفس المنتج قد يأخذ نموذجين مختلفين حسب الفريق.

Factor 3 — Characteristics of the customer

Verbatim (Mall p.120):

Characteristics of the customer: If the customer is not quite familiar with computers, then the requirements are likely to change frequently as it would be difficult to form complete, consistent, and unambiguous requirements. Thus, a prototyping model may be necessary to reduce later change requests from the customers.

العميل: لو مو فاهم بالحاسبات، المطلوبات راح تتغير كثير (صعب تتكتب كاملة وواضحة) → Prototyping ضروري لتقليل طلبات التغيير لاحقاً. هذا يربط بملف 03 (النموذج يتعلم المطلوبات).

Three-factor summary table (paraphrase from §2.6.1, p.120): | Factor | Pulls toward | |---|---| | Product: small services | Agile | | Product: product / embedded | Iterative waterfall | | Product: object-oriented | Evolutionary | | Team: experienced in similar software | Iterative waterfall (even embedded) | | Team: entirely novice | Prototyping (even simple DPA) | | Customer: unfamiliar with computers | Prototyping (requirements drift) |

5. Requirements and Risk — The Two Drivers (integration of Mall + Sommerville)

The plan names **requirements and risk** as *the two drivers*. Reading Mall's three factors against Sommerville's framing shows they are **two lenses on the same decision**, not a disagreement.

Sommerville's lens (supporting, p.46):

Verbatim (Sommerville p.46):

For some systems, such as critical systems, a very structured development process is required. For business systems, with rapidly changing requirements, a less formal, flexible process is likely to be more effective.

Verbatim (Sommerville p.46):

Sometimes, software processes are categorized as either plan-driven or agile processes. Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan. In agile processes, which I discuss in Chapter 3, planning is incremental and it is easier to change the process to reflect changing customer requirements.

Verbatim (Sommerville p.46):

As Boehm and Turner (2003) discuss, each approach is suitable for different types of software. Generally, you need to find a balance between plan-driven and agile processes.

عدسة Sommerville: أنظمة حرجة → عملية منظّمة جداً (plan-driven). أنظمة تجارية بمطلوبات متغيرة → عملية مرنة (agile). وينصح بتوازن بين الاثنين. هذي عدسة "نوع النظام + استقرار المطلوبات".

How the two lenses map onto the two drivers:

Requirements driver — *clarity & volatility*. Stable, well-understood requirements → iterative waterfall (Mall p.118; Sommerville's "structured process" for critical systems). Volatile / not-yet-known requirements → prototyping (learn them), evolutionary / agile (absorb them), incremental (slice them). This is exactly the series' spine: requirements always change; the model you pick is your *answer* to that.

Risk driver — *identifiability & emergence*. Risks known upfront → prototyping (Mall p.119). Risks emerge during development → spiral (Mall p.119; Unit 06). Low risk + well-understood → iterative waterfall. High/changing risk + large → spiral.

No contradiction [Foundational Knowledge / Standard Concept]: Mall's three factors (product / team / customer) and Sommerville's two axes (critical↔business, plan-driven↔agile) are **complementary descriptions of the same choice**. Requirements-and-risk is the *underlying axis*; Mall's three factors are the *concrete knobs* a lead turns. Showing both, not merging them, is the correct reading.

6. Worked Selection Scenarios

6.1 The Payroll Case — Exercise Q40 (Mall p.130)

Verbatim scenario (Mall p.130):

Assume that a software development company is already experienced in developing payroll software and has developed similar software for several customers (organisations). Assume that the software development company has received a request from a certain customer (organisation), which was still using manually processing of its pay rolls. For developing a payroll software for this organisation, which life cycle model should be used? Justify your answer.

Worked answer (derived/applied reasoning — not a direct quote; built from §2.6 + §2.6.1):

Payroll is a **well-understood, stableRequirements** problem — the rules (tax, deductions, grades) are standard and known. → satisfies Mall's "iterative waterfall is suitable only for well-understood problems" (p.118).

The company is **experienced** in similar payroll software. → satisfies Mall's team factor: "If the development team is experienced in developing similar software, then even an embedded software can be developed using an iterative waterfall model" (p.120). The same logic applies a fortiori to an ordinary data-processing application.

The customer was manual, so there is no volatile pre-existing automated system to disrupt; requirements can be captured up front.

Conclusion: Iterative waterfall is the fit. (This is the canonical textbook answer to Q40; Mall does not print a model name in the exercise text, so the justification above is the applied reasoning, anchored to §2.6 and §2.6.1.)

حالة الرواتب (Q40): شركة **خبيرة** برمجيات رواتب، والمطلوبات **مفهومة ومستقرة** (قوانين ضريبة واستقطاع معروفة)، والعميل كان يدوي. → النموذج المناسب هو **Iterative Waterfall** (مفهوم جيداً + فريق خبير). هذا الجواب المشتق من §2.6 و §2.6.1.

6.2 The Galaxy Case — Why Spiral (Mall p.118, Case study 2.2)

Verbatim (Mall p.118):

Galaxy Inc. undertook the development of a satellite-based communication between mobile handsets that can be anywhere on the earth... However, the risks in the project are many, including determining how the calls among the satellites can be handed-off when they are themselves revolving at a very high speed. In the absence of any published material and availability of staff with experience in development of similar products, many of the risks cannot be identified at the start of the project and are likely to crop up as the project progresses. The software would require several million lines of code to be written. Galaxy Inc. decided to deploy the spiral model for software development after hiring highly qualified staff... The project was successfully completed after five years.

حالة Galaxy: مخاطر **ما تتوقع بالبداية** (تسليم المكالمات بين أقمار سريعة الدوران، وما أكو خبرة سابقة) + ملايين السطور. هذا بالضبط موقف الـ Spiral (ملف 06): مخاطر تطلع أثناء التطوير → Spiral. أنجز المشروع بـ 5 سنوات. مثال واقعي يثبت قاعدة "المخاطر المتأخرة = Spiral".

6.3 The selection guide in one picture

Choosing a Model: No Winner, Only Fit

Three factors decide · two drivers dominate · and the documentation always ends up reading like waterfall (Mall §2.6, pp.118–121)

THE THREE FACTORS THAT INFLUENCE SELECTION (MALL p.120)

1 · The product

- How well are the requirements understood?
- How stable are they likely to stay?
- How large and how complex is the system?
- What is the quality and reliability demand?

2 · The development team

- Size — small teams suit agile, large teams need structure
- Experience and skill level of the members
- How geographically distributed is the team?
- How familiar are they with the technology?

3 · The customer

- Can they take part actively and continuously?
- Can a representative sit inside the team?
- Will they accept an incremental or evolving system?
- Are they contracting formally, or collaborating?

THE SELECTION GUIDE — WHICH SITUATION CALLS FOR WHICH MODEL

SITUATION	MODEL	WHY IT FITS
Requirements fully stable, low risk	Classical / Iterative waterfall	Nothing will change, so freeze everything up front
Requirements unclear, risk identifiable	Prototyping	Build something to discover what is actually wanted
Requirements keep changing in the field	Evolutionary / Incremental	Let the system itself absorb the change
Large project, requirements well understood	Incremental	Deliver in slices; the size rules out a single build
Schedule is the binding constraint	RAD	Parallel feature teams, component reuse, time-box
Risk is the dominant factor, large scale	Spiral	Risk becomes the control variable of the project
Object-oriented, need one adoptable framework	Unified Process	Phases × workflows, deployment treated as part of the process
Small team, requirements move fast	Agile / XP / Scrum	Two-to-three-week increments, customer inside the team
Mission- or safety-critical system	A very structured process	Reliability first: plan-driven beats flexible (Sommerville p.46)

⚠ THE TRAP THAT CATCHES EVERYONE — why the final documentation reads like waterfall

Even when a project is built incrementally, iteratively or agilely, the **final documentation is written as if the classical waterfall had been used**.
The reason: the customer and the maintainer need a **linear, readable account** of what the system is — not the actual order in which it was discovered. (Mall p.121)

Sources: Mall, Fundamentals of Software Engineering (4th ed.), §2.6–§2.6.1, pp.118–121 · Sommerville, Software Engineering (9th ed.), p.46.

Model selection: the three factors, the situation-to-model guide, and the documentation trap

كيف تقرأ الرسم — وهذا جواب سؤال «أي نموذج أختار؟»:

معناه الهندسي	العنصر في الرسم
العوامل الثلاثة (Mall p.120): المنتج · فريق التطوير · العميل. الثلاثة معاً تحدد النموذج — مو عامل واحد	الصناديق الثلاثة العليا
الخريطة العملية: تسعة سيناريوهات وكل واحد ونموذجه المفضل	جدول «SITUATION» → «MODEL»
الاستثناء الأهم: الأنظمة الحرجة/الأمنية ما تستخدم Agile — تحتاج عملية منظّمة (Sommerville p.46)	الصف الأخير (حرج)
فخ التوثيق: حتى لو بنيت incrementally، الوثائق النهائية تُكتب كأنك استخدمت waterfall	الصندوق الأصفر السفلي

الخلاصة البصرية: الاختيار مو تفضيل شخصي — هو دالة لثلاثة متغيّرات. ولو سُئلت «أي نموذج يناسب هذا المشروع؟» — لا تجاوب بنموذج قبل ما تحدد: هل المطلوبات مستقرة؟ شنو الخطر الأكبر؟ شنو حجم الفريق؟

7. Why Final Documentation Is Written As If Waterfall (Summary, p.121)

Even when a team uses spiral, agile, or anything else, the **paper trail** is 标准化 to waterfall shape.

Verbatim (Mall p.121 — Summary):

Even though an organisation may follow whichever life cycle model is appropriate to a project, the final document should reflect as if the software was developed using the classical waterfall model. This makes it easier for the maintainers to understand the software documents.

مهما كان النموذج المستخدم فعلياً، الوثيقة النهائية تُكتب وكأنها **Waterfall**. السبب: المشرفين/الصيانة يفهمون وثائق الـ Waterfall بسهولة. هذا يجاب على سؤال التمرين Q44 (Mall p.130): "لماذا يجب أن توصف الوثائق النهائية البرنامج وكأنه طُور بـ *Waterfall*؟" — الجواب هو "ليسهل على المشرفين فهمها".

Why this matters for "fit": the *execution* model is chosen for fit; the *documentation* model is chosen for **legibility to future maintainers**. They are deliberately decoupled. A team can run agile internally yet ship waterfall-shaped documents.

8. Source notes

Primary (Mall §2.6 + §2.6.1 + Summary + Case study + Q40/Q53/Q55): carries the entire decision framework — the per-model suitability list, the three selection factors, the customer-viewpoint argument, the spiral/waterfall documentation rule, and the worked exercises. This file is **Mall-primary by design** of the plan.

Supporting (Sommerville §2.1 p.46): supplies the *second lens* — critical-systems-need-structure vs business-systems-need-flexibility, and the plan-driven↔agile axis with the "balance" caution (Boehm & Turner 2003). Sommerville does **not** enumerate the three factors; that detail is Mall-only. The two are complementary, shown side by side in §5, not merged.

What adds nothing here: Pressman and Agarwal are not cited for Unit 09 — the selection decision is fully covered by Mall + Sommerville. No gap results; this is intentional scope, not a coverage note gap.

Coverage note: Sommerville's contribution is only the ~half-page framing on p.46; it gives the *axes* but no factor enumeration and no per-model suitability list. The granular "which model for which situation" content is entirely Mall's. A reader wanting Sommerville-only guidance on selection would find it thin — hence the Mall-primary weighting.

Anchor accuracy note: Sommerville's supporting passage prints the page label "29" in the book but sits at **PDF physical page 46** (the number used here). Mall has no printed labels; its cited pages are PDF physical pages (idx+1), which match the plan's "p.118 / p.130–131".

Retrieval set (9 items — questions and answers together)

RS-09-01 Why does Mall say no single life-cycle model is "the best"?

The classical waterfall is the "basic model" and all others are "embellishments" of it, yet even the most-used model (iterative waterfall) is "suitable only for well-understood problems" and not for very large or high-risk projects. Each model fits a specific context, so the right question is fit, not superiority. [Mall p.118]

RS-09-02 What are Mall's three factors that influence SDLC selection?

(1) Characteristics of the software/product (small services → agile; product/embedded → iterative waterfall; OO → evolutionary); (2) Characteristics of the development team (experienced → even embedded via iterative waterfall; novice → even simple DPA via prototyping); (3) Characteristics of the customer (unfamiliar with computers → requirements drift → prototyping). [Mall p.120]

RS-09-03 According to Mall, which model is favoured for small services, for product/embedded software, and for object-oriented development?

Small services → agile; product/embedded → iterative waterfall; object-oriented → evolutionary. [Mall p.120]

RS-09-04 How does team skill level change the model choice?

An experienced team can use iterative waterfall even for embedded software; an entirely novice team may need prototyping even for a simple data-processing application. Skill level can override product type. [Mall p.120]

RS-09-05 How does customer computer-familiarity affect the choice?

An unfamiliar customer makes requirements likely to change (hard to form complete, consistent, unambiguous requirements), so a prototyping model is used to reduce later change requests. [Mall p.120]

RS-09-06 When is prototyping used vs the spiral model?

Prototyping when risks are few and identifiable at project start; spiral when risks are hard to anticipate at the beginning but likely to crop up as development proceeds. [Mall p.119]

RS-09-07 From the customer's viewpoint, why do incremental/evolutionary approaches reduce resentment and "trauma"?

In monolithic (waterfall) development customer confidence drops because no working software is visible and delays are announced in technical slang. Incremental/evolutionary lets the customer experiment with working software earlier, adjust gradually to the new system (less trauma), and avoids a large upfront capital outlay (pay per increment as affordable). [Mall p.119]

RS-09-08 Payroll case (Mall Q40): a company experienced in payroll software must build payroll software for a customer still using manual processing. Which model, and why?

Iterative waterfall. Payroll is a well-understood, stable-requirements problem, and the team is experienced in similar software — both conditions that Mall ties to iterative waterfall (suitable for well-understood problems; experienced teams can use it even for harder products). [Derived from Mall p.118 + p.120; scenario Mall p.130]

RS-09-09 Why are final documents written as if the software was developed with the classical waterfall model, regardless of the actual model used?

To make it easier for the maintainers to understand the software documents. Execution model is chosen for fit; documentation model is standardized to waterfall for legibility. [Mall p.121; exercise Q44 p.130]

9. Page anchors

Mall: p.118 (Case study 2.2 Galaxy → spiral; §2.6 start: waterfall as basic model, iterative waterfall most-used-but-limited) · p.119 (§2.6: prototyping, evolutionary, spiral-as-meta-model; prototyping-vs-spiral risk test; customer viewpoint — confidence/trauma/capital) · p.120 (§2.6.1: the three selection factors — product, team, customer) · p.121 (Summary: final docs written as if waterfall) · p.130 (Exercise Q40 payroll scenario; Q44 why docs as waterfall) · p.131 (Exercise Q53 three factors; Q55 important factors).

Sommerville: p.46 (§2.1: critical systems need structured process; business systems with changing requirements need flexible process; plan-driven vs agile defined; Boehm & Turner balance caution).

End of Unit 09. Next: **Unit 10 — Master Comparison and Exam Bank** (all models side by side + the consolidated retrieval bank).

Master Studio · Advanced Software Engineering (CS504) · University of Wasit